

UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**

SPECIALIZAREA INFORMATICĂ

Lucrare de disertație

**Proiectarea, implementarea și securizarea unei infrastructuri cloud
automatizate în Microsoft Azure utilizând Bicep, Packer și Ansible**

Absolvent

Valentin TIȚA

Coordonator științific

Prof. univ. dr. Paul-Andrei Păun

București, iunie 2026

Rezumat

Această lucrare prezintă proiectarea, implementarea și securizarea unei infrastructuri cloud automatizate în Microsoft Azure, folosind Bicep, Packer, Ansible și Azure DevOps. Tema este tratată ca studiu de caz aplicativ, construit în jurul relației dintre un beneficiar fictiv, SC MEDIA SRL, și un furnizor IT, SC IT SECURITY SRL. Am urmărit să arăt cum poate fi transformată o cerință organizațională relativ obișnuită - găzduire site web, bază de date, server de fișiere, acces administrativ controlat, monitorizare și securizare - într-o soluție reproductibilă, documentată și verificabilă.

Contribuția lucrării constă în integrarea coerentă a mai multor practici moderne: infrastructură de tip cod (eng. Infrastructure as Code – IaC), imagini standardizate (eng. golden images), managementul configurațiilor (eng. Configuration management), gestionarea parolelor, monitorizare și testare. Am pus accent nu doar pe tehnologie, ci și pe dimensiunea umană a proiectului: comunicarea cu beneficiarul, reducerea asimetriei de cunoaștere, predarea infrastructurii și menținerea continuității operaționale. Rezultatul este o arhitectură potrivită pentru un client mic sau mediu, suficient de robustă pentru a demonstra principii profesionale, dar suficient de clară pentru a fi explicată și extinsă.

Cuprins

Rezumat.....	2
1. Introducere	1
1.1 Contextul și motivația lucrării.....	1
1.2 Obiective, contribuție și metodologie	2
1.3 Structura lucrării.....	4
2. Fundamente teoretice și poziționare.....	4
2.1 Calcul în cloud și responsabilitate operațională.....	4
2.2 Infrastructură drept cod, DevOps și reproductibilitate.....	5
2.3 Rolul tehnologiilor utilizate	7
2.4 Observabilitate și limbaj comun.....	9
3. Analiza cerințelor și contextul organizațional.....	10
3.1 Actorii scenariului	10
3.2 Cerințe funcționale și non-funcționale	11
3.3 Dimensiunea umană a cerințelor	12
3.4 Riscuri inițiale și criterii de acceptanță	13
4. Arhitectura soluției	14
4.1 Viziune generală.....	14
4.2 Separarea responsabilităților tehnice.....	15
4.3 Fluxuri de livrare și organizarea depozitului.....	16
4.4 Compromisuri arhitecturale.....	17
5. Implementarea practică	17
5.1 Mediul de lucru, secretele și imaginile.....	17
5.2 Definirea infrastructurii cu Bicep.....	19
5.3 Configurarea serviciilor cu Ansible	21

5.4 Scripturi operaționale și documentare	23
5.5 Administrarea schimbării și mentenanța	24
6. Securizare, monitorizare și validare	25
6.1 Modelul de securitate	25
6.2 Gestionarea parolelor și controlul accesului	27
6.3 Monitorizare și răspuns operațional	28
6.4 Testare și validare.....	29
6.5 Limite ale validării	31
7. Concluzii și direcții de dezvoltare	31
7.1 Rezultatele obținute	31
7.2 Contribuții și valoare practică	32
7.3 Limitări și dezvoltări viitoare	33
7.4 Concluzie finală.....	34
Bibliografie.....	36
Anexa 1: Codul sursă al proiectului	38
Anexa 2. Glosar de termeni tehnici utilizați.....	39
Anexa 3. Matrice sintetică de verificare.....	44
Anexa 4: Arhitectura cloud	45
Anexa 5: Firul de execuție.....	46

1. Introducere

1.1 Contextul și motivația lucrării

Infrastructura este un spațiu în care se întâlnesc tehnologia, responsabilitatea și încrederea. Într-o companie mică, tehnologia nu este analizată în primul rând prin rafinamentul ei intern, ci prin efectele pe care le produce în activitatea zilnică. Un server de fișiere indisponibil poate bloca o echipă creativă, iar un site public oprit poate afecta credibilitatea și de ce nu, întreaga activitate a unei companii.

Am scris această lucrare pornind de la convingerea că infrastructura nu este doar un decor tehnic, ci o formă de grijă față de oamenii care depind de ea. De aceea, atunci când descriu Bicep, Packer, Ansible sau Azure, nu urmăresc doar să enumăr instrumente, ci să arăt felul în care le-am folosit pentru a construi ordine, continuitate și încredere într-un scenariu organizațional credibil.

Scenariul de lucru are la bază o companie fictivă, dar plauzibilă, SC MEDIA SRL, care are nevoie de o infrastructură în cloud completă și de un partener IT de încredere care să facă totul posibil și totodată să ofere mentenanță și îmbunătățiri continue. Furnizorul, SC IT SECURITY SRL, trebuie să livreze soluția într-o manieră repetabilă și transparentă. Am ales această relație deoarece multe proiecte reale de infrastructură nu apar în companii cu departamente IT mari, ci în organizații care externalizează expertiza și care au nevoie de încredere, nu doar de resurse create într-un portal de servicii cloud. Compania SC MEDIA SRL, care oferă servicii de PR, Marketing și Media clienților săi, neavând cunoștințe în domeniul IT, este sfătuită să evite achiziția echipamentelor costisitoare și greu de întreținut necesare desfășurării activității (servele, aparate de securizare a rețelei și datelor de tip firewall, licențe dedicate, etc.). În acest sens, cloud-ul devine mai mult decât o colecție de servicii disponibile la cerere. El devine un mod de a organiza dependențele unei companii și reprezintă soluția cea mai fiabilă, care asigură îndeplinirea tuturor cerințelor și nevoilor companiei, cât și continuitatea acesteia. Cine poate intra, unde poate intra, ce se întâmplă când ceva se oprește, cine primește alerta, cum se reface mediul, cum se păstrează parolele, cum se documentează schimbările: toate acestea sunt întrebări tehnice, dar și întrebări de organizare și responsabilitate, pe care SC Media SRL trebuie doar să le adreseze, urmând a fi adresate de către personalul calificat al integratorului IT.

În scenariul prezentat, Bicep descrie/lansează infrastructura în Microsoft Azure, Packer standardizează imaginile sistemelor de operare folosite, Ansible aplică starea dorită asupra serverelor prin modificările pe care le aduce acestora într-un mod idempotent, iar Azure DevOps oferă versionare, execuție și trasabilitate. Împreună, ele reduc dependența de intervenții manuale și transformă infrastructura într-un obiect verificabil, standardizat și ușor de interpretat de către oricare specialist ce primește acces la codul sursă. Totodată, trebuie recunoscut faptul că a avea totul în cloud are de multe ori și implicații de natură psihologică (uneori pozitive, alteori negative): beneficiarul nu mai vede serverul, nu mai știe unde este fizic, nu poate „merge la el” și nu poate deduce starea infrastructurii din semne vizibile. În locul materialității aparatelor apar metrice, loguri, alerte, dashboard-uri și rapoarte. Așadar, în relația de colaborare furnizor-beneficiar, integratorul are datoria de a „traduce” sistemele abstracte de cloud în ceva inteligibil și plin de sens pentru client [1], [2, pp. 3-6], [3, pp. 5-6], [4, pp. 20-31].

Voi încerca să folosesc pe cât posibil termeni tehnici în limba română, fără a forța însă formulări artificiale sau chiar ridicole. Unele denumiri consacrate, precum Bicep, Packer, Ansible, Key Vault, DevOps sau pipeline, rămân necesare pentru recunoașterea lor în documentația tehnică. Totuși, acolo unde limba română oferă o formulare firească, prefer exprimări precum flux automatizat, depozit de cod, copie de siguranță, server de fișiere sau întărire a securității.

1.2 Obiective, contribuție și metodologie

Din punct de vedere metodologic, am tratat proiectul ca pe o succesiune de decizii, nu ca pe o simplă execuție. Fiecare alegere tehnică răspunde unei întrebări: ce risc reduce, ce operație simplifică, ce dovadă produce sau ce dependență elimină. Această abordare m-a ajutat să păstrez legătura dintre scopul academic al lucrării și utilitatea ei practică.

Contribuția personală se vede mai ales în integrare. Instrumentele folosite sunt consacrate, însă modul în care sunt combinate, delimitate și explicate formează valoarea proiectului. Am încercat să evit atât prezentarea superficială a tehnologiilor, cât și excesul de detalii care ar transforma lucrarea într-un manual de utilizare.

Obiectivul principal al lucrării este proiectarea unei infrastructuri cloud automatizate, securizate și verificabile pentru un client mic sau mediu. Din acest obiectiv derivă mai multe obiective specifice: definirea infrastructurii prin cod, standardizarea imaginilor de mașini virtuale, configurarea serviciilor prin *playbook*-uri Ansible, gestionarea secretelor prin mecanisme specializate, monitorizarea stării sistemului și validarea soluției prin teste funcționale și de securitate.

Contribuția mea nu constă în inventarea unui limbaj sau a unui algoritm nou, ci în integrarea riguroasă a unor instrumente consacrate într-un scenariu complet, realist și argumentat. Am urmărit să arăt cum se iau deciziile, nu doar cum se rulează comenzile. De aceea, lucrarea include și analiza compromisurilor: de ce o topologie clasică a rețelelor virtuale (flat VNet) este suficientă pentru scenariul propus, de ce accesul administrativ este concentrat într-un server dedicat, de ce unele configurări aparțin imaginii Packer, iar altele rămân în Ansible.

Metodologia este aplicativă. Am pornit de la cerințe, am proiectat arhitectura, am ales tehnologiile, am descris implementarea și am validat rezultatul prin criterii de acceptanță. În același timp, am raportat proiectul la literatura despre infrastructură prin cod (IaC), securitate în IaC și reproductibilitate. Această combinație între fundamentare și implementare este potrivită pentru o disertație de master orientată spre practică.

Automatizarea infrastructurii apare în acest proiect ca răspuns la o problemă veche: fragilitatea lucrului făcut manual. O configurație aplicată manual poate funcționa perfect în ziua în care este realizată, dar devine dificil de explicat, repetat și auditat. Infrastructură ca cod schimbă această situație deoarece mută deciziile într-un spațiu persistent. O subrețea, o regulă în grupul de securitate de rețea (Network Security Group -NSG), o mașină virtuală, un Seif de chei (Key Vault) sau o asocierie de monitorizare nu mai sunt doar obiecte create în portal într-un mod manual, ci declarații păstrate în fișiere. Această mutare are o valoare tehnică, dar și una culturală: infrastructura devine discutabilă, revizibilă și transmisibilă. Pentru un furnizor de servicii IT, automatizarea înseamnă și profesionalizarea livrării. În loc să construiască fiecare proiect ca pe o lucrare unică, furnizorul poate crea un nucleu reutilizabil. Nu este vorba despre a trata toți clienții la fel, ci despre a păstra o bază de calitate constantă: aceleași principii de securitate, aceleași proceduri de implementare, aceleași verificări, aceleași mecanisme de revenire la o stare anterioară funcțională (rollback). Automatizarea reduce totodată dependența de improvizația făcută manual. În ecosistemele IT complexe din zilele noastre, improvizația nu este doar inefficientă, ci foarte riscantă.

1.3 Structura lucrării

Lucrarea este organizată în șapte capitole. Capitolul 1 introduce tema, motivația, obiectivele și metodologia. Capitolul 2 prezintă fundamentele teoretice: calcul în cloud, infrastructură prin cod, DevOps, imagini imuabile, managementul configurațiilor și observabilitate. Capitolul 3 analizează cerințele beneficiarului și dimensiunea organizațională a proiectului. Capitolul 4 descrie arhitectura soluției. Capitolul 5 prezintă implementarea practică. Capitolul 6 reunește securizarea, monitorizarea și testarea. Capitolul 7 formulează concluziile, limitările și direcțiile de dezvoltare.

2. Fundamente teoretice și poziționare

2.1 Calcul în cloud și responsabilitate operațională

Cloud-ul este adesea prezentat prin avantajele sale economice, însă în această lucrare îl privesc mai ales ca pe un mediu de organizare, întrucât diferențele de costuri impuse de achiziția echipamentelor fizice de către beneficiar în contrast cu costurile lor în cloud ar fi atât de mare încât nici să le compar n-are avea rost. În cloud, resursele pot fi create rapid, dar tocmai această rapiditate cere reguli. Fără convenții de nume, etichete (taguri), politici, documentație și monitorizare, mediul cloud poate deveni mai greu de controlat decât o infrastructură locală mică, sau, în alte cuvinte, un haos funcțional.

Pentru un furnizor IT, cloud-ul schimbă și modelul de prestare a serviciilor. În locul unei intervenții punctuale pe un server local, furnizorul administrează un ecosistem: identități, costuri, acces, regiuni, imagini, jurnale și politici. Această schimbare justifică folosirea automatizării, deoarece administrarea manuală devine repede vulnerabilă la eroare și uitare.

Un aspect esențial al calculului în cloud este schimbarea felului în care organizația înțelege proprietatea asupra infrastructurii. Resursele nu mai sunt echipamente fizice aflate într-un spațiu local, ci servicii configurabile. Această schimbare oferă flexibilitate, dar poate crea și impresia greșită că responsabilitatea tehnică dispare. În realitate, responsabilitatea se mută și se împarte altfel.

Pentru scenariul lucrării, modelul Infrastructură drept Serviciu (Infrastructure as a Service - IaaS) este potrivit deoarece permite demonstrarea completă a procesului de infrastructură: rețea, mașini

virtuale, imagini, sisteme de operare, configurare, securizare, copii de siguranță și monitorizare. Dacă aș fi ales exclusiv servicii complet gestionate, o parte din complexitate ar fi fost ascunsă [1].

În același timp, alegerea IaaS obligă la prudență. Fiecare mașină virtuală are nevoie de actualizări, reguli de acces, politici de securitate și monitorizare. Cloud-ul oferă un cadru elastic, dar nu garantează singur un sistem bun, întrucât responsabilitățile sunt împărțite. De aceea, am asociat utilizarea Azure cu practici de automatizare și documentare.

Calculul în cloud permite furnizarea de resurse IT la cerere: mașini virtuale, rețele, stocare, baze de date, identitate, monitorizare și servicii de securitate. Pentru un client mic, avantajul imediat este evitarea investițiilor inițiale în echipamente, dar avantajul mai profund este flexibilitatea: mediul poate fi creat, modificat, extins și, la nevoie, refăcut mai rapid decât într-o infrastructură locală tradițională sau, de ce nu, în alt mediu cloud, oferit de un alt furnizor (de exemplu Amazon Web Services sau Google Cloud Platform).

Totuși, flexibilitatea cloud-ului nu elimină responsabilitatea. În modelul IaaS, pe care se bazează lucrarea, așa cum am menționat anterior, furnizorul cloud administrează platforma de bază, dar sistemele de operare, serviciile instalate, regulile de acces, actualizările și monitorizarea rămân responsabilități ale echipei care proiectează soluția. Tocmai de aceea, cloud-ul nu trebuie confundat cu simplitatea automată. El oferă mijloace, dar calitatea rezultatului depinde de arhitectură, disciplină și verificare.

Ca pasionat de muzică, pot asemăna automatizarea cu o formă de orchestrare. Nu în sens decorativ, ci structural. Fiecare instrument are intrarea lui, fiecare tehnologie are momentul ei, iar ordinea contează. Packer pregătește tema de bază: imaginile. Bicep intră cu arhitectura: rețea, mașini virtuale, identitate, copii de siguranță, monitorizare. Ansible aduce variațiile concrete: servicii, fișiere, configurații, întărire a securității. Azure DevOps ține ritmul schimbărilor, iar monitorizarea ascultă dacă ansamblul se dezacordează.

2.2 Infrastructură drept cod, DevOps și reproductibilitate

Reproductibilitatea are două sensuri în lucrare. Primul este tehnic: mediul poate fi recreat pornind de la artefacte controlate. Al doilea este academic: execuțiile sunt detaliate astfel încât creează o lizibilitate sporită și demonstrează felul de funcționare și valoarea adusă de toate

tehnologiile folosite. O lucrare aplicativă devine mai solidă atunci când rezultatul nu se bazează doar pe afirmații, ci pe pași, fișiere și criterii de verificare.

Infrastructura drept cod introduce și o formă de critică înainte de execuție. Verdet, Hamdaqa, Da Silva și Khomh observă că adoptarea practicilor de securitate în IaC este inegală și că infrastructura scrisă în cod poate rămâne vulnerabilă dacă nu este însoțită de controale explicite [5]. Dacă o regulă de rețea este prea permisivă, ea poate fi observată într-o recenzie. Dacă un parametru sensibil (precum o parolă) este introdus direct în cod, poate fi separat. Dacă o resursă nu respectă convenția de nume, poate fi corectată. Acest lucru mută o parte din control din zona incidentului în zona proiectării.

Practica DevOps nu garantează automat calitate. Un flux automatizat poate implementa rapid și o greșeală (tehnologiile care permit gestionarea infrastructurii și serviciilor prin cod nu cunosc intențiile și dorințele specialistului IT, așadar ele funcționează atâta timp cât parametrii furnizați și codul sunt corecte). Din acest motiv, am asociat automatizarea cu validare, documentare și prudență. Automatizarea este utilă când execută o decizie bună; dacă decizia este greșită, ea doar multiplică problema mai eficient.

Un mare avantaj constă în faptul că Infrastructura drept Cod aduce infrastructura în zona textului verificabil. Această idee este importantă nu doar tehnic, ci și academic: un artefact scris poate fi analizat, criticat, comparat și în cele din urmă îmbunătățit. Un click în portal se consumă în momentul execuției; o modificare în cod rămâne în istoric. Această diferență explică de ce IaC este mai mult decât automatizare rapidă.

În proiect, reproductibilitatea este tratată ca valoare practică. Nu urmăresc doar să creez o dată infrastructura, ci să pot explica și relua pașii. Scripturile numerotate, fișierele de parametri, imaginile versionate și *playbook*-urile Ansible formează o memorie tehnică a proiectului. Această memorie reduce riscul ca mediul să depindă exclusiv de autor.

DevOps adaugă acestei memorii un flux de lucru. Depozitul de cod, validările, execuția controlată și jurnalele transformă schimbarea într-un proces. Pentru un furnizor IT, acest lucru are și valoare comercială: nu livrează doar intervenții izolate, ci un mod de lucru care poate fi repetat și adaptat.

Infrastructura drept cod reprezintă practica de a descrie resursele infrastructurii prin fișiere text, versionabile și verificabile. În loc ca un specialist IT să creeze manual resurse în portal, intenția

arhitecturală este exprimată în cod. Această schimbare are efecte importante: modificările pot fi revizuite, diferențele pot fi urmărite, iar mediul poate fi recreat mai ușor.

Literatura de specialitate tratează IaC ca pe o extindere a practicilor de dezvoltare software către infrastructură. Rahman, Mahdavi-Hezaveh și Williams arată că defectele din artefactele IaC pot afecta direct mediile de dezvoltare și de implementare [6]. Chiari, De Pascalis și Pradella discută analiza statică a IaC ca răspuns la riscurile de securitate și fiabilitate [7]. Aceste perspective susțin ideea că fișierele Bicep, HCL sau YAML nu sunt simple auxiliare, ci artefacte critice care trebuie citite, verificate și documentate.

DevOps completează această logică printr-o cultură a colaborării, automatizării și feedback-ului. În lucrare, DevOps înseamnă cod versionat, pași clari, validări, jurnale și posibilitatea de a reconstitui istoricul. Zhao, Niu, Huang și Zhang discută reproductibilitatea în contexte Cloud DevOps [8], iar această idee se transferă natural către infrastructura proiectului: dacă mediul este definit prin cod și parametri, el poate fi repetat și verificat.

2.3 Rolul tehnologiilor utilizate

Bicep, Packer și Ansible formează o triadă utilă pentru proiecte în care infrastructura trebuie să fie clară și repetabilă/scalabilă. Bicep lucrează cu resursele cloud, Packer cu starea inițială a sistemelor (indiferent de mediul în care se lucrează), iar Ansible cu configurarea serviciilor (indiferent de mediul de lucru). Această împărțire este ușor de explicat și are avantajul că separă momentele diferite ale „ciclului de viață” (din eng. lifecycle) [1], [3, pp. 5-6], [4, pp. 20-31].

Un risc al unor astfel de proiecte este acumularea de instrumente. Am încercat să evit această capcană prin justificarea fiecărui instrument. Dacă un instrument nu reduce un risc, nu clarifică un proces sau nu produce o valoare operațională, el nu ar trebui introdus doar pentru că este modern. În lucrare, fiecare tehnologie are un rol delimitat.

Această delimitare este importantă și pentru predare. Un alt specialist IT trebuie să știe unde caută o problemă: în codul Bicep, în imaginea Packer, în rolul Ansible, în fluxul Azure DevOps sau în jurnalele Azure Monitor. Fără această hartă, proiectul ar fi mai greu de menținut decât sugerează automatizarea lui.

Am separat tehnologiile după nivelul la care produc valoare. Bicep este specializat în resurse Azure și descrie relațiile dintre componentele infrastructurii. Packer este potrivit pentru starea inițială a

sistemelor de operare, deoarece creează imagini controlate. Ansible este potrivit pentru configurații care trebuie aplicate și reaplicate după pornirea serverelor, iar pe parcurs se întrebuintează pentru întreținerea continuă a acestora (orchestrare de actualizări, remediere vulnerabilități prin aplicarea codului de remediere pe toată infrastructura, etc).

Această separare previne o problemă frecventă: folosirea unui singur instrument pentru toate rolurile. Dacă aș pune prea mult în imaginea Packer, fiecare schimbare mică ar cere reconstruirea imaginii. Dacă aș lăsa totul în Ansible, timpul de configurare ar crește și baza inițială ar fi mai puțin uniformă. Dacă aș crea resurse Azure prin pași manuali, reproductibilitatea și omogenitatea ar scădea.

Azure DevOps are rolul de cadru al schimbării. El nu face arhitectura corectă de unul singur, dar permite execuția ordonată și păstrarea istoricului. Într-un proiect profesional, acest istoric este esențial atunci când trebuie explicat ce s-a schimbat, cine a schimbat și cu ce rezultat.

Bicep este limbajul prin care am descris toate resursele Azure. El oferă o sintaxă mai lizibilă decât șabloanele ARM (Azure Resource Manager) JSON și permite modularizarea infrastructurii. În proiect, Bicep descrie rețeaua virtuală, subrețelele, mașinile virtuale, IP-urile publice, grupurile de securitate, identitățile, sistemele copiilor de siguranță (eng. backup), extensiile și componentele de monitorizare [1], [2, pp. 3-6].

Packer este folosit pentru construirea imaginilor standardizate, conform nevoilor specifice ale companiei. O imagine pregătită anterior reduce variația dintre servere și scurtează timpul de configurare după pornire. Într-un mediu profesional, această standardizare este importantă deoarece evită diferențele subtile dintre mașini create manual. Nu am pus însă toate configurările în Packer, deoarece ar fi rigidizat schimbările. Am păstrat în imagine baza comună, iar comportamentul specific l-am tratat cu Ansible. Definițiile imaginilor create cu Packer sunt publicate într-o Galerie de Calcul Azure (Azure Compute Gallery) și sunt automat incrementate pe măsură ce o nouă variantă a fiecărui sistem de operare este stocată în galerie [3, pp. 5-6], [9].

Ansible aplică modificări pentru a atinge starea dorită asupra serverelor. El configurează servicii precum nginx, WordPress, MySQL, SMB, reguli de securitate și verificări operaționale. Avantajul său este idempotența: un playbook bine scris poate fi rulat ori de câte ori se dorește fără a produce modificări. Azure DevOps leagă aceste etape prin depozit de cod (eng. repository), fluxuri

automatizate și istoric al schimbărilor. Azure Monitor și Log Analytics completează ansamblul prin observabilitate [4, pp. 20-31], [10, pp. 27, 61].

Tehnologie	Rol în lucrare	Valoare adăugată
Bicep	Declară resursele Azure și relațiile dintre ele	Reproductibilitate, modularitate, trasabilitate
Packer	Construiește imagini standardizate	Reducerea variației inițiale dintre servere
Ansible	Configurează serviciile și aplică starea dorită	Idempotență, mentenanță, separare de responsabilități, orchestrare actualizări
Azure DevOps	Versionare și execuție controlată	Audit al schimbărilor și livrare repetabilă
Azure Monitor	Colectează metrice, jurnale și alerte	Feedback operațional și intervenție informată

2.4 Observabilitate și limbaj comun

Observabilitatea este una dintre diferențele dintre o demonstrație reușită și un sistem operabil. O infrastructură poate fi creată corect, dar fără jurnale și alerte rămâne greu de înțeles în timp. Monitorizarea nu este o anexă tehnică, ci mecanismul prin care furnizorul poate răspunde informat atunci când apare o problemă.

Limbajul comun are aceeași importanță. Clientul nu trebuie să devină specialist Azure, dar trebuie să poată înțelege de ce anumite decizii au fost luate. Dacă îi spun doar că există (grupuri de securitate al rețelelor (NSG-uri), Seif de chei (Key Vault) sau identități gestionate (managed identities), risc să produc distanță. Dacă explic riscurile pe care aceste componente le reduc, infrastructura devine mai inteligibilă și capătă sens în ochii tuturor.

O infrastructură automatizată nu este completă fără observabilitate. Automatizarea creează mediul, dar monitorizarea arată dacă mediul funcționează corect în timp. În lipsa jurnalelor și alertelor, furnizorul poate declara că implementarea a reușit, fără să poată demonstra comportamentul ulterior al sistemului. De aceea, am inclus monitorizare pentru resurse, servicii și semnale de securitate. Alerte pentru procesor (CPU), disc, servicii, porturi și tentative de autentificare eșuate

crează un minim operațional sănătos, suficient pentru a demonstra maturitate fără a transforma proiectul într-o platformă organizațional supradimensionată.

Un alt element teoretic important este limbajul comun dintre client și furnizor. Un client non-tehnic nu trebuie să cunoască sintaxa Bicep sau detaliile WinRM (Windows Remote Management – protocolul de administrare automatizată/de la distanță al sistemelor de operare Windows), dar trebuie să înțeleagă de ce accesul este restricționat, de ce parolele nu sunt păstrate în fișiere obișnuite și de ce schimbările trebuie făcute prin cod. Pentru mine, această traducere este o parte a profesionalismului tehnic.

3. Analiza cerințelor și contextul organizațional

3.1 Actorii scenariului

Am păstrat caracterul fictiv al companiilor pentru a putea discuta liber scenariul, dar am construit nevoile lor din situații plauzibile, extrase din realitatea domeniului de activitate. O companie de comunicare lucrează cu materiale digitale, termene, clienți și imagine publică. De aceea, stabilitatea serviciilor nu este un detaliu tehnic, ci o condiție a muncii zilnice.

Furnizorul IT are o responsabilitate dublă. Pe de o parte, trebuie să proiecteze și să implementeze corect. Pe de altă parte, trebuie să explice suficient de clar încât clientul să nu perceapă infrastructura ca pe o cutie neagră totală. Această responsabilitate de traducere este una dintre competențele importante ale specialistului modern.

SC MEDIA SRL este definită ca o companie de comunicare și marketing care are nevoie de infrastructură digitală stabilă pentru activități editoriale, campanii, materiale creative și colaborare internă. Nu am tratat compania ca pe un pretext tehnic, ci ca pe un beneficiar cu ritm de lucru, presiuni și riscuri. Pentru o astfel de organizație, indisponibilitatea unui site sau a unui server de fișiere poate afecta direct relația cu clienții.

SC IT SECURITY SRL este furnizorul care proiectează, implementează și predă infrastructura. Rolul său nu este doar execuția tehnică, ci și traducerea cerințelor de business în decizii verificabile. Furnizorul trebuie să gestioneze asimetria de cunoaștere: el înțelege tehnologia, clientul înțelege

consecințele asupra activității sale. O livrare matură apare când cele două perspective se întâlnesc și totodată când integratorul reușește să înțeleagă în întregime nevoile și preferințele clientului, reușește să ofere cele mai valoroase soluții și le traduce cu succes în termeni uzuali pe care îi pot înțelege persoane non-tehnice.

3.2 Cerințe funcționale și non-funcționale

Cerințele non-funcționale sunt adesea scoase din context, sau, mai bine precizat, din planul pe care integratorul i-l propune clientului. Acest lucru a creat de multe ori confuzie și în unele cazuri a dus chiar la deteriorarea relațiilor dintre părți, deoarece costurile finale n-au mai reflectat realitatea promisă. De exemplu, Seiful de Chei (Key Vault) nu apare în arhitectură pentru că nu este un serviciu Azure cunoscut, ci pentru că răspunde unei cerințe de protejare a parolilor. Azure Monitor nu apare pentru completitudine, ci pentru observabilitate. Ansible nu apare pentru varietate tehnologică, ci pentru configurare repetabilă. Majoritatea cerințelor non-funcționale nu sunt direct observabile de către beneficiar. Am creat un tabel care categorisește o parte dintre ele:

Cerință	Materializare în proiect
Securitate	Acces minim necesar, NSG-uri restrictive, Key Vault, Ansible Vault (pentru stocarea parolilor folosite de Ansible), întărirea securității resurselor, fail2ban.
Repetabilitate	Deploy prin Bicep, imagini Packer și <i>playbook</i> -uri Ansible idempotente.
Audit	Cod versionat, <i>pipeline</i> -uri, jurnale text și HTML, rapoarte.
Cost	VM-uri B-series, Log Analytics (treapta de taxare gratuită), doar două IP-uri publice persistente.
Operabilitate	<i>Jumphost</i> (mașină virtuală situată „la marginea ecosistemului” pe care se conectează specialistul IT pentru a lucra în interiorul mediului cloud) cu instrumente preinstalate și verificări automate.

Cerințele funcționale sunt relativ ușor de enumerat, dar cerințele non-funcționale dau maturitatea proiectului. Un site care răspunde este necesar, dar nu suficient. Trebuie să știu cine îl poate administra, cum se salvează configurațiile, cum se detectează problemele și cum se evită expunerea inutilă a serviciilor interne.

Din acest motiv, am tratat securitatea, reproductibilitatea și documentarea ca cerințe, nu ca îmbunătățiri opționale. În proiectele reale, tocmai aceste aspecte sunt amânate când timpul este scurt, iar efectele apar mai târziu: intervenții greu de urmărit, parole pierdute, reguli deschise temporar și uitate (ce ulterior duc la breșe majore de securitate și chiar la scurgeri de date!), costuri neexplicate.

Cerințele funcționale includ găzduirea unui site public, existența unui sistem de gestionare a conținutului (din end. Content Management System – CMS), o bază de date, un server de fișiere, un server de acces administrativ (denumit jumphost) și mecanisme de monitorizare. Cerințele non-funcționale privesc securitatea, reproductibilitatea, mentenanța, costurile proporționale, documentarea și posibilitatea de a recrea mediul. Am acordat atenție acestor cerințe deoarece ele decid dacă soluția este doar demonstrabilă sau cu adevărat operabilă.

Categorie	Cerință	Răspuns în soluție
Funcțională	Site public și CMS	Server web cu nginx/WordPress, expus controlat
Funcțională	Bază de date	MySQL pe server dedicat, accesibil doar intern
Funcțională	Spațiu de fișiere	Server Windows pentru partajare controlată

3.3 Dimensiunea umană a cerințelor

Asimetria de cunoaștere dintre client și furnizor este inevitabilă. Furnizorul cunoaște tehnologia, clientul cunoaște impactul asupra propriei activități. Dacă una dintre perspective o ignoră pe cealaltă, soluția devine dezechilibrată. De aceea, am încercat să păstrez cerințele ancorate în consecințe, nu doar în componente.

Încrederea nu apare din promisiunea generală că furnizorul se ocupă de tot. Ea apare din dovezi: proceduri, jurnale, teste, documente, reguli și explicații. Când clientul vede că există o ordine a implementării și că schimbările sunt trasabile, dependența de furnizor devine mai sănătoasă.

Un proiect de infrastructură externalizat nu eșuează doar prin erori tehnice. El poate eșua și prin neînțelegere, lipsă de comunicare, așteptări implicite sau dependență de un singur specialist. De aceea, am inclus criteriile de acceptanță care privesc nu doar funcționarea serviciilor, ci și claritatea predării: ce primește clientul, cum se solicită o schimbare, cum se raportează un incident și ce limite are soluția.

Încrederea clientului este construită prin proceduri: pași de implementare, jurnale, rezultate de test, reguli de acces și explicații. Nu este suficient ca furnizorul să spună că mediul este securizat; trebuie să poată arăta cum este securizat. Nu este suficient ca infrastructura să funcționeze; ea trebuie să poată fi operată și de altcineva decât autorul inițial.

3.4 Riscuri inițiale și criterii de acceptanță

Criteriile de acceptanță trebuie să fie suficient de concrete pentru a putea fi verificate. Nu este suficient să spun că mediul este securizat; trebuie să arăt că serverele interne nu sunt expuse public, că secretele sunt protejate, că accesul administrativ este limitat și că există monitorizare. O întărire a securității sitului web al companiei SC Media SRL va veni însoțită de analiza și rezultatele testelor de securitate disponibile pe Internet atât înainte de aplicarea actualizărilor cât și după. Aceste criterii transformă afirmațiile în verificări.

Am preferat o listă de riscuri realistă în locul unei liste exhaustive. Scopul nu este să acopăr toate scenariile posibile, ci să arăt că proiectul tratează riscurile principale ale unui mediu cloud pentru o companie mică: expunere, secrete, configurare manuală, lipsă de observabilitate și predare incompletă. Pentru fiecare risc, am urmărit un control tehnic sau procedural. De exemplu, expunerea este controlată prin NSG-uri, parolele prin Seiful de Chei (Key Vault și Ansible Vault), iar reproductibilitatea prin cod și scripturi de execuție.

Risc	Impact posibil	Măsură de reducere
Acces administrativ expus	Compromiterea serverelor	Server de acces administrativ, doar IP-ul specialistului IT permis, NSG-uri restrictive

Risc	Impact posibil	Măsură de reducere
Secrete în fișiere obișnuite	Scurgere de credențiale	Key Vault, Ansible Vault, identități gestionate
Configurare manuală	Mediu greu de refăcut	Bicep, Packer, Ansible, depozit de cod
Lipsă monitorizare	Incidente detectate târziu	Azure Monitor, alerte, jurnale centralizate
Predare incompletă	Dependență de autor	Documentație, glosar, criterii de acceptanță

4. Arhitectura soluției

4.1 Viziune generală

Arhitectura poate fi citită ca o hartă a expunerii. Componentele publice sunt puține și justificate, iar componentele interne rămân protejate. Acest principiu este important deoarece multe incidente nu apar din lipsa unor instrumente avansate, ci din expuneri simple și necontrolate: porturi deschise, parole slabe, servicii administrative accesibile direct.

Am păstrat în arhitectură și ideea de resurse persistente. IP-urile publice, secretele și anumite elemente de monitorizare nu trebuie tratate ca detalii trecătoare, deoarece schimbarea lor poate afecta DNS, accesul, documentația și încrederea clientului. Stabilitatea infrastructurii înseamnă și stabilitatea acestor repere.

Arhitectura generală urmărește să păstreze sistemul lizibil. Am preferat o topologie care ar putea fi operată de o echipă mică și explicată unui client fără a pierde rigoarea tehnică. Mașinile virtuale au roluri clare: web, bază de date, fișiere și administrare. Separarea rolurilor ajută atât securitatea, cât și mentenanța. Dacă un serviciu are probleme, pot identifica mai repede zona relevantă. Dacă o regulă de acces trebuie modificată, pot vedea ce componentă este afectată.

Arhitectura propusă urmărește un echilibru între claritate, securitate și cost. Am ales o rețea virtuală Azure împărțită în subrețele funcționale, cu expunere publică limitată. Doar serverul web și serverul de acces administrativ au IP public, iar celelalte resurse rămân accesibile doar intern. Această alegere reduce suprafața de atac și face mai ușor de explicat traseul accesului.

Nu am ales o arhitectură *hub-spoke* sau multiregiune deoarece scenariul nu justifică această complexitate. Pentru un client mic sau mediu, o topologie clasică de rețea segmentată prin subrețele și grupuri de securitate este mai potrivită: poate fi înțeleasă, operată și extinsă. Am preferat o soluție care să poată crește ulterior, în locul unei arhitecturi spectaculoase, dar disproporționate.

Componentă	Rol	Expunere
Server web	Găzduire site/CMS	Public controlat
Server bază de date	Persistență date aplicație	Intern
Server de fișiere	Partajare documente	Intern
Server acces administrativ	Punct de intrare pentru administrare	Public restricționat
Key Vault	Stocarea secretelor	Acces prin identități și politici
Log Analytics	Colectare jurnale și metrice	Serviciu gestionat Azure

4.2 Separarea responsabilităților tehnice

Un avantaj al separării este că permite evoluția independentă. Pot îmbunătăți imaginile fără să rescriu rețeaua, pot modifica roluri Ansible fără să schimb topologia și pot ajusta monitorizarea fără să reconstruiesc întregul mediu. Această independență reduce riscul ca o schimbare locală să producă efecte laterale greu de controlat.

Separarea responsabilităților ajută și în discuția cu beneficiarul. Clientul nu are nevoie de detalii despre sintaxă, dar poate înțelege că infrastructura are straturi: platformă, sistem de operare, aplicație, securitate și monitorizare. Când aceste straturi sunt explicate, proiectul devine mai puțin opac.

Separarea responsabilităților este una dintre deciziile cele mai importante ale proiectului. Ea previne amestecul dintre infrastructură, imagine și configurație. În loc să am o colecție de pași greu de despărțit, am niveluri tehnice distincte. Această ordine face proiectul mai ușor de testat și de întreținut. De asemenea, separarea ajută la scalare. Dacă integratorul ar administra mai mulți clienți, ar putea reutiliza anumite module Bicep, anumite imagini Packer și anumite roluri Ansible, adaptând doar parametrii și serviciile specifice. Astfel, experiența acumulată se transformă în artefacte, nu rămâne doar în memoria echipei.

Așadar, am separat responsabilitățile pentru a evita confuzia dintre nivelurile infrastructurii. Bicep definește resursele Azure; Packer pregătește imaginile de sistem; Ansible configurează serviciile după pornirea mașinilor; Azure DevOps orchestrează pașii; Azure Monitor observă comportamentul. Această separare ajută la diagnostic: dacă o resursă nu apare, caut în Bicep; dacă imaginea este greșită, caut în Packer; dacă serviciul nu pornește, verific Ansible.

Această împărțire este și o decizie de mentenanță. Un sistem în care un singur script face totul poate părea eficient la început, dar devine greu de întreținut. Prin separarea etapelor, fiecare componentă poate fi testată, explicată și înlocuită mai ușor. Într-un proiect predat unui client sau unei echipe, această claritate are valoare practică.

4.3 Fluxuri de livrare și organizarea depozitului

Un proiect devine greu de operat atunci când ordinea reală a acțiunilor există doar în mintea autorului. De aceea, fluxurile și scripturile trebuie să comunice singure ordinea: ce se pregătește, ce se construiește, ce se implementează și ce se testează.

Această disciplină are și un rol de audit. Dacă apare o problemă, se poate verifica mai ușor etapa relevantă. Jurnalele de construire a imaginilor, rezultatele implementării Bicep și ieșirile Ansible nu sunt simple fișiere auxiliare, ci dovezi ale procesului. Ele pot fi folosite pentru diagnostic și pentru justificarea deciziilor în fața beneficiarului.

Organizarea depozitului este o parte a arhitecturii, nu doar a administrării fișierelor. Modul în care sunt așezate directoarele spune cititorului cum trebuie înțeles proiectul. Dacă fișierele sunt amestecate, atunci și responsabilitățile devin neclare. Dacă sunt separate coerent, proiectul poate fi parcurs mai ușor.

Fluxurile de livrare susțin aceeași idee. Ele creează un drum de la cod la infrastructură. Într-un mediu real, acest drum ar include aprobări și controale suplimentare. În lucrare, el demonstrează principiul: schimbarea nu este o intervenție izolată, ci o succesiune controlată de pași.

Depozitul de cod (eng. repository) este organizat pe zone de responsabilitate: fișiere Bicep, fișiere Packer, roluri Ansible, scripturi operaționale, documentație și jurnale. Această organizare nu este doar estetică. Ea arată cum trebuie citit proiectul și reduce dependența de memoria autorului. Un evaluator sau un operator poate urmări fluxul de la inițializarea secretelor până la validarea finală.

Fluxurile automatizate sunt gândite pentru pași distincți: construirea imaginilor, implementarea infrastructurii și configurarea serviciilor. Pentru producție, aceste fluxuri ar trebui completate cu aprobări, politici de ramură și conexiuni de serviciu cu privilegii minime. În forma proiectului, ele demonstrează principiul unei livrări repetabile și auditabile.

4.4 Compromisuri arhitecturale

Compromisurile trebuie raportate la context. O soluție mai complexă nu este automat mai bună. Pentru SC MEDIA SRL, costul, capacitatea de operare și claritatea sunt la fel de importante ca robustețea. O arhitectură foarte sofisticată, dar greu de administrat, ar putea produce mai mult risc decât o arhitectură mai simplă, dar bine documentată.

Totuși, am notat direcțiile de evoluție deoarece soluția nu trebuie privită ca final universal. Azure Bastion, hub-spoke, integrare SIEM (Security Information and Resource Management) și politici de conformitate pot deveni necesare pe măsură ce organizația crește. Lucrarea fixează o etapă realistă, nu o maturitate tehnică finală.

Am inclus compromisurile pentru că ele arată maturitatea proiectului. O lucrare care prezintă doar decizii ideale riscă să pară lipsită de aplicabilitate. În infrastructură, fiecare alegere are costuri: simplitatea poate limita extinderea, securitatea poate reduce comoditatea, standardizarea poate cere timp inițial.

Topologia rețelelor clasice (VNets & Subnets) este un exemplu. Ea nu este soluția potrivită pentru toate organizațiile, dar este justificabilă aici. Dacă scenariul ar evolua către mai multe medii, conexiuni hibride sau cerințe de conformitate, aş propune o arhitectură mai segmentată. Pentru faza analizată, însă, proporționalitatea este o virtute.

Orice arhitectură presupune compromisuri. Am folosit un server de acces administrativ în locul Azure Bastion pentru a păstra controlul asupra scenariului și pentru a demonstra automatizarea configurării.

5. Implementarea practică

5.1 Mediul de lucru, secretele și imaginile

În scenariul proiectului, diferența dintre Linux și Windows a fost importantă. Serverele Linux sunt mai naturale pentru configurare prin SSH (Secure Shell), în timp ce serverele Windows

cer atenție suplimentară prin WinRM. Această diferență arată de ce infrastructurile reale sunt rareori uniforme. Un furnizor trebuie să gestioneze tehnologii diferite fără să piardă coerența procesului.

Imaginile personalizate reduc timpul de configurare, dar nu trebuie confundate cu o soluție totală. Ele pregătesc o bază, nu înlocuiesc managementul configurațiilor. Am păstrat această distincție deoarece ea ajută la mentenanță: schimbările frecvente rămân în Ansible, iar schimbările de bază intră în ciclul de viață al imaginii.

Un alt motiv pentru folosirea imaginilor este controlul calității. Dacă fiecare server pornește dintr-o imagine diferită, diferențele devin greu de explicat. Dacă pornește dintr-o bază comună, erorile sunt mai ușor de reprodus. În infrastructură, capacitatea de a reproduce o eroare este aproape la fel de importantă ca rezolvarea ei.

În practică, etapa de pregătire este ușor de subestimat. Dacă mediul local, autentificarea, secretele și versiunile instrumentelor nu sunt clare, problemele apar mai târziu și sunt greu de atribuit.

Construirea imaginilor Packer are și rol de documentare operațională. Ea arată ce se consideră că trebuie să existe înainte ca serverul să primească rolul său final. Așadar imaginea oferă fundația, Ansible configurează funcția.

Gestionarea parolelor este una dintre zonele în care disciplina tehnică se vede rapid. În proiectele grăbite, parolele ajung ușor în fișiere locale, conversații sau documentații. Am evitat această practică și am tratat parolele ca resurse cu regim special. O parolă nu este doar un șir de caractere; este o promisiune că accesul va fi limitat. A o pune într-un fișier obișnuit, într-un mesaj sau într-un depozit de cod înseamnă a transforma această promisiune într-o vulnerabilitate.

Azure Key Vault oferă mecanismul tehnic, dar decizia de a-l folosi exprimă o maturitate de proiectare. Primul script operațional, numit ``0-bootstrap-keyvault.ps1``, creează resursele persistente necesare: grupul de resurse Azure persistent și Key Vault-ul în care sunt stocate parolele de infrastructură. Mai târziu în firul de execuție, pe serverul de acces administrativ (jump host), scriptul ``create-ansible-vault.sh`` preia secretele prin Managed Identity și construiește fișierul ``group_vars/all/vault.yml``, criptat cu Ansible Vault [11].

Imaginile Packer reduc diferențele dintre servere. În loc să pornesc de la mașini virtuale complet brute, am construit o bază comună. Această bază nu rezolvă totul, dar scurtează drumul până la configurarea finală. Pentru mine, imaginea standardizată este echivalentul unui punct de plecare curat.

Implementarea începe cu pregătirea mediului de lucru și a secretelor. În loc să includ parole sau chei în fișiere, am folosit *Azure Key Vault* ca loc legitim pentru parole și *Ansible Vault* pentru folosirea lor în configurări. Această alegere reduce riscul scurgerii de credențiale și susține principiul conform căruia informațiile sensibile nu trebuie să circule informal.

Dezavantajul Packer este timpul mai mare de construire, dar acest cost este justificat deoarece imaginea poate fi reutilizată.

Principalele detalii ale imaginilor create cu Packer:

Image definition	Sistem	Conținut principal
imgdef-ubuntu2204	Ubuntu 22.04 LTS base	Update sistem, pachete comune, auditd, hardening SSH, timezone.
imgdef-ubuntu2204-jumphost	Ubuntu 22.04 LTS jumphost	XFCE, xRDP, Ansible, Azure CLI, pywinrm, Remmina, instrumente de administrare.
imgdef-winsrvr2022	Windows Server 2022	WinRM preconfigurat, Visual C++ Redistributable, hardening de bază.

Imaginile sunt publicate privat în Azure Compute Gallery. Această alegere permite versionare și reutilizare internă, fără a expune imaginile în afara organizației. Pentru un furnizor de servicii, galeria devine o bibliotecă de standarde operaționale [12].

5.2 Definirea infrastructurii cu Bicep

Bicep este limbajul prin care arhitectura devine declarație. Această trecere de la diagramă la cod este esențială. O diagramă poate explica intenția, dar codul creează realitatea. Într-un proiect matur, cele două trebuie să se susțină reciproc: diagrama arată sensul, codul arată implementarea.

În fișierele Bicep, modularizarea ajută la separarea logicii. Un modul pentru rețea, unul pentru mașini virtuale și unul pentru monitorizare sunt mai ușor de citit decât un fișier unic foarte lung. Această organizare sprijină și reutilizarea. Un furnizor poate prelua un modul, îl poate adapta și îl poate folosi la un alt client [1], [2, pp. 3-6].

Bicep contribuie și la guvernarea prin faptul că face vizibile deciziile. Regiunile, dimensiunile, etichetele (tags), regulile de acces și identitățile nu sunt create accidental. Ele apar în cod și pot fi discutate. Aceasta este una dintre calitățile majore ale infrastructurii drept cod: decizia tehnică devine citibilă. Simplul fapt că infrastructura este declarată și versionată reprezintă un pas important față de administrarea manuală. Lucrarea arată această trecere de la intervenție la model.

Un avantaj al Bicep este că permite folosirea unei sintaxe apropiate de intenția arhitecturală. Resursele, modulele și parametrii pot fi citite mai ușor decât într-un fișier JSON foarte amplu.

Validarea înainte de implementare are rol de protecție. Analiza Bicep de tip *what-if* (simularea execuției) arată ce urmează să se schimbe și poate preveni surprize. Într-un mediu real, această etapă ar trebui completată cu revizuire și aprobări pentru producție. În lucrare, ea demonstrează că IaC nu este tratat ca pe o execuție oarbă [1].

Parametrizarea este importantă deoarece separă logica de valori. Numele resurselor, dimensiunile, locațiile și anumite opțiuni pot fi schimbate fără rescrierea modulelor. Această separare face soluția mai potrivită pentru reutilizare la clienți similari.

Bicep transformă arhitectura în declarații executabile. Am folosit module pentru a descrie rețeaua, resursele de calcul, regulile de securitate, identitățile și monitorizarea. Înainte de aplicare, validarea și analiza *what-if* oferă o previzualizare a schimbărilor, reducând riscul de modificări neintenționate.

Această abordare face ca portalul Azure să nu mai fie sursa principală de adevăr. Portalul rămâne util pentru observare și diagnostic, dar deciziile arhitecturale trebuie să fie în cod. Dacă cineva dorește să înțeleagă de ce există o subrețea sau o regulă NSG, răspunsul trebuie să fie într-un fișier versionat, nu într-o memorie personală a unui om.

Fișierul `main.bicep` funcționează ca orchestrator la nivel de subscripție. El apelează module specializate pentru grupul/grupurile de resurse, rețelistică, NSG, compute, *Key Vault*, monitorizare, politici, IP-uri publice persistente, AMA (Azure Monitor Agent – agentul serviciului

de monitorizare al Azure ce colectează informațiile de pe fiecare sistem), RBAC (Role Based Access Control – Controlul accesului pe bază de roluri) și politici de acces. Parametrii sunt separați în fișiere `.bicepparam` atât pentru producție cât și pentru dezvoltare.

Modul Bicep	Responsabilitate
networking.bicep	Creează VNet și subrețelele mgmt, prod, dev.
nsg.bicep	Definește regulile de acces și interziceri implicite.
compute.bicep	Creează VM-uri, NIC-uri (Network Interface Card- interfete virtuale de rețea), OS disks și runCommands pentru WinRM.
monitoring.bicep ama.bicep	/ Log Analytics, alerte și Azure Monitor Agent.
policy.bicep	Politici Azure pentru guvernare și conformitate.
kv-access-policy.bicep	Acces Managed Identity jumhost la parolele din Key Vault.

Scriptul `2-deploy-teardown-bicep.ps1` adaugă un strat practic peste Bicep: detectează IP-ul public al administratorului/specialistului IT, rulează validări, execută *what-if*, solicită confirmare și generează jurnale HTML. Astfel, specialistul nu rulează manual o serie lungă de comenzi, ci folosește un flux controlat.

5.3 Configurarea serviciilor cu Ansible

Dacă Bicep creează corpul infrastructurii, Ansible îi dă comportament. VM-urile devin server web, server CMS, server de aplicații, bază de date sau server de fișiere prin configurațiile aplicate după provisionare. Această etapă este apropiată de viața concretă a sistemului, pentru că aici apar serviciile pe care oamenii le folosesc indirect. Pentru client, Ansible nu este vizibil. Dar efectele sale sunt vizibile: servicii configurate consistent, întărire a securității aplicată, WordPress instalat, MySQL pregătit, fișiere partajate disponibile [4, pp. 20-31], [10, pp. 27, 61].

Pentru serverul web, Ansible poate instala serviciile, poate copia fișiere de configurare și poate verifica starea proceselor. Pentru baza de date, poate gestiona instalarea, configurarea inițială și reguli de acces. Pentru serverul de fișiere, poate pregăti partajările și permisiunile. Fiecare rol traduce o nevoie de business într-o stare tehnică.

Această traducere este importantă deoarece arată cum devine concretă cerința clientului. Clientul nu cere „un *playbook*”, ci un site disponibil, date protejate și fișiere accesibile. Ansible este doar mecanismul prin care aceste rezultate devin repetabile. În corpul lucrării am păstrat explicația rezultatului, nu listingurile complete.

Idempotența este criteriul care diferențiază un *playbook* matur de o simplă listă de comenzi. Dacă reluarea produce efecte nedorite, automatizarea devine periculoasă și execuția nu asigură idempotență. Dacă reluarea confirmă sau corectează starea dorită, automatizarea devine instrument de mentenanță. Acesta este motivul pentru care testarea idempotenței apare în validare.

Configurarea prin Ansible reduce dependența de pași manuali. Dacă trebuie instalat un serviciu, modificată o regulă sau verificată o stare, *playbook*-ul păstrează logica. Această păstrare este importantă deoarece intervențiile manuale reușite nu lasă întotdeauna o urmă suficientă pentru viitor.

În același timp, Ansible trebuie scris cu grijă. Un *playbook* prost organizat poate deveni la fel de greu de menținut ca un set de comenzi manuale. De aceea, rolurile, variabilele, inventarul și secretele trebuie separate. Această disciplină nu este doar tehnică, ci și documentară.

Ansible completează infrastructura cu comportament. O mașină virtuală nu este suficientă prin simpla ei existență; ea trebuie să devină server web, server de bază de date sau server de fișiere. *Playbook*-urile descriu această transformare într-un mod repetabil.

Am ales să păstrez configurările de serviciu în Ansible deoarece ele se pot schimba mai frecvent decât baza sistemului. Versiuni, opțiuni, reguli, servicii și verificări pot fi ajustate fără reconstruirea imaginii. Actualizările sistemelor de operare și remedierile problemelor de securitate intră de asemenea sub incidența îndeletnicirilor Ansible. Această flexibilitate a motorului este utilă în mentenanță.

După crearea resurselor, Ansible configurează serviciile. Pe serverul web instalez și configurez componentele necesare pentru site și CMS. Pe serverul de baze de date configurez MySQL și reguli

de acces. Pe serverul de fișiere configurez partajarea controlată. Pe serverul de acces administrativ pregătesc punctul de intrare pentru operații tehnice. În fiecare caz, scopul este ca starea dorită să fie descrisă explicit.

Ansible este potrivit pentru această etapă deoarece permite reluarea configurării. Dacă o parte a sistemului trebuie verificată sau reparată, *playbook*-urile pot fi rulate din nou. Această proprietate este importantă în mentenanță, unde intervențiile manuale repetate produc ușor diferențe între medii.

Ansible este rulat de pe serverul de acces administrativ (jumphost), după execuția scriptului `3-deploy-ansible-to-jumphost.ps1`, care copiază toate fișierele de cod ale Ansible pe serverul de acces administrativ. Inventarul principal este dinamic, prin `azure\_rm.yml`, cu autentificare `auth\_source: msi`, ceea ce elimină nevoia de credențiale Azure locale. Grupurile sunt derivate din etichete (tags) și nume de resurse, iar conectivitatea se face prin SSH pentru Linux și WinRM pentru Windows. Rolurile și funcțiile sale sunt: [13]

Rol Ansible	Funcție
common	Baseline Linux: pachete, NTP (Network time protocol), audit, firewall.
nginx	Reverse proxy, TLS, security headers și rate limiting.
wordpress	WordPress, PHP-FPM, WP-CLI și conexiune la MySQL.
mysql	MySQL 8.0 pe Windows, utilizatori, baze de date, TDE, audit.
fileserver	SMB shares, ACL-uri NTFS, dezactivare SMBv1.
hardening / fail2ban / modsecurity	Măsuri de securitate avansată și întăriri de securitate.

5.4 Scripturi operaționale și documentare

Jurnalele generate de scripturi au valoare de audit și de comunicare. Ele permit reconstruirea parcursului: ce s-a rulat, când, cu ce rezultat. Într-o situație de incident, această urmă

reduce timpul pierdut cu presupuneri. Într-o situație de predare, ea arată beneficiarului că livrarea nu a fost improvizată. Prin configurarea scripturilor de execuție să producă și versiuni HTML ale jurnalelor, rezultatele sunt și mai ușor de observat de către oricine. Scripturile operaționale sunt puntea dintre codul tehnic și utilizarea lui de către un operator. Ele reduc riscul ca cineva să ruleze pașii în ordine greșită sau cu parametri lipsă.

Scripturile numerotate au rolul de a transforma implementarea într-un traseu clar: inițializare, construire imagini, implementare infrastructură, transfer Ansible (execuția mutându-se apoi pe *jumphost*), configurare cu Ansible și testare. Numerotarea nu este o simplă convenție; ea reduce ambiguitatea pentru operator (foarte util în situația în care specialistul se schimbă). În loc să ghicească ordinea pașilor, acesta o vede în numele și structura fișierelor.

5.5 Administrarea schimbării și mentenanța

Mentenanța este locul în care se vede dacă proiectul a fost gândit pentru viața reală. Un mediu poate fi creat spectaculos o singură dată, dar adevărata lui valoare apare când trebuie actualizat, reparat sau recreat. De aceea, am păstrat în lucrare ideea de continuitate.

Pentru furnizor, mentenanța repetabilă este și o condiție de creștere. Dacă fiecare client cere intervenții complet manuale, furnizorul nu poate scala fără să piardă controlul. Dacă există șabloane, proceduri și teste, experiența acumulată devine reutilizabilă.

Administrarea schimbării este relevantă deoarece infrastructura nu rămâne neschimbată. Clientul poate cere un serviciu nou, o regulă diferită, o dimensiune mai mare sau o modificare de acces. Dacă aceste schimbări sunt făcute manual, istoria lor se pierde ușor. Dacă sunt făcute prin cod, ele devin parte din memoria proiectului.

Mentenanța cere și o cultură a verificării periodice. Nu este suficient ca sistemul să fie corect la predare. Trebuie urmărite actualizările, alertele, spațiul pe disc, accesul, secretele și documentația. În acest sens, proiectul propune o bază, nu un final definitiv.

O infrastructură bună trebuie să poată fi schimbată fără teamă. În modelul propus, modificările se fac prin cod, se versionază și pot fi revizuite. Aceasta nu elimină nevoia de judecată, dar reduce improvizația. Dacă se schimbă dimensiunea unei mașini virtuale, o regulă de acces sau o configurație de serviciu, modificarea trebuie să lase urme.

Mentenanța include actualizări, rezolvarea vulnerabilităților, rotația parolelor, verificarea alertelor, revizuirea regulilor de acces și testarea periodică. Automatizarea nu elimină responsabilitatea umană, ci o sprijină. Un sistem automatizat prost înțeles poate produce erori rapid; un sistem automatizat și documentat poate reduce presiunea asupra operatorilor.

Ce tareEtapă	Instrument principal	Rezultat urmărit
Inițializare parole	Key Vault / scripturi	Parole persistente și acces controlat
Construire imagini	Packer	Imagini standardizate Linux/Windows
Creare resurse	Bicep	Rețea, VM-uri, identități, monitorizare
Configurare servicii	Ansible	Servicii instalate, configurare, întăriri ale sistemelor de operare și stare dorită
Validare	Scripturi/teste	Dovezi funcționale și operaționale

Jurnalele de execuție (log) text sunt utile pentru specialiști, dar mai greu de urmărit pentru un beneficiar sau pentru persoane fără expertiză tehnică. De aceea, proiectul generează și jurnale HTML, cu secțiuni mai ușor de citit. Această alegere are valoare dublă: ajută la depistarea problemelor la execuție și arată într-un mod mai „prietenos” ce s-a executat.

Din perspectiva mentenanței, jurnalele păstrează istoria execuțiilor și pot ajuta la identificarea momentului în care o schimbare a produs efecte neașteptate. Ele sunt mai ales utile în scripturile care orchestrează comenzi externe, unde eroarea poate veni din Azure CLI, Packer, SSH, SCP, Ansible sau PowerShell.

6. Securizare, monitorizare și validare

6.1 Modelul de securitate

La nivel de aplicație, securitatea depinde de configurări precum TLS (Transport Layer Security), reguli pentru serverul web, protecții împotriva încercărilor brute de autentificare ilicită și limitarea accesului către baza de date. La nivel de sistem, depinde de utilizatori, servicii active, actualizări și jurnale. La nivel de platformă, depinde de rețea, identitate și politici Azure. Am încercat să arăt cum aceste niveluri se susțin reciproc.

Un proiect de această dimensiune nu poate garanta securitate absolută. Poate însă demonstra o gândire de securitate: identificarea riscurilor, aplicarea controalelor, verificarea lor și recunoașterea limitelor. Această gândire este mai importantă decât enumerarea unui număr mare de instrumente.

Securitatea trebuie să fie demonstrabilă. Nu este suficient să afirm că mediul este securizat; trebuie să existe controale și verificări. În lucrare, securitatea se observă în arhitectura de rețea, în gestionarea parolelor, în accesul administrativ, în întărirea sistemelor și în monitorizare.

Am evitat să prezint securitatea ca pe o stare finală. Ea este un proces care trebuie menținut. O regulă bună astăzi poate deveni insuficientă mâine, iar un serviciu sigur la instalare poate deveni vulnerabil dacă nu este actualizat. De aceea, securitatea este legată încontinuu de mentenanță.

Securitatea în straturi pornește de la premisa că oamenii și sistemele pot greși. O parolă poate fi compromisă, o regulă poate fi configurată prea permisiv, un serviciu poate avea o vulnerabilitate. Mai multe controale reduc șansa ca o singură eroare să producă un incident major.

Am distribuit controalele pe mai multe niveluri: rețea, identitate, parole, sistem de operare, aplicație și monitorizare. Această distribuție este mai potrivită decât încrederea într-o singură barieră. Dacă o barieră eșuează, celelalte pot limita impactul.

Modelul *Defense in Depth* (apărare în profunzime) presupune că niciun control nu este suficient singur. Rețeaua limitează expunerea, identitățile controlează accesul, *Key Vault* gestionează parolele, imaginile sunt întărite, Ansible aplică reguli de configurare, iar monitorizarea oferă semnale despre funcționare și posibile incidente.

La nivel de rețea, doar serviciile care trebuie să fie vizibile sunt expuse public. Serverele interne rămân protejate prin subrețele și NSG-uri. Accesul administrativ este concentrat într-un server dedicat și restricționat. Această decizie reduce suprafața de atac și simplifică auditarea accesului.

Dintre întăririle de securitate aplicate pe infrastructură, merită amintite următoarele:

- Serverul web folosește nginx ca reverse proxy, certificate Let's Encrypt, TLS 1.2/1.3, HSTS, OCSP stapling și security headers. ModSecurity cu OWASP CRS adaugă un nivel de protecție pentru atacuri web uzuale, iar rate limiting-ul reduce riscul de abuz pe endpoint-uri sensibile precum ``wp-login.php`` sau ``/api/``.
- fail2ban monitorizează tentativele eșuate și poate bloca automat IP-uri după un număr configurat de eșecuri. Rolul de întărire a securității SSH elimină algoritmi slabi și păstrează

suite criptografice moderne precum curve25519 și ChaCha20-Poly1305. În ansamblu, accesul administrativ devine mai greu de exploatat prin atacuri de volum sau negocieri criptografică slabă.

- MySQL pe Windows Server este configurat cu utilizatori dedicați pentru WordPress, API și monitorizare. Configurația elimină utilizatori anonimi, dezactivează 'local_infile', activează audit logging și folosește TDE pentru datele la latente. Această separare de conturi și funcții limitează mișcarea laterală în cazul compromiterii unei aplicații.
- Azure Monitor Agent colectează date din sistemele de operare și le trimite către Azure Monitor/Log Analytics. În proiect sunt definite alerte pentru brute-force SSH, brute-force Windows, servicii oprite, CPU ridicat, spațiu disc critic și verificări de funcționare. Health scripturile rulează periodic și scriu evenimente ușor de interogat prin KQL [14].

Demonstrații practice ale securității:

Script demo	Scop
demo-1-rate-limiting.sh	Depășirea limitei pe endpoint-uri sensibile produce răspuns 429.
demo-2-fail2ban.sh	Tentative SSH eșuate duc la blocarea IP-ului.
demo-3-ssh-hardening.sh	Algoritmii slabi SSH sunt respinși.
demo-4-modsecurity.sh	SQLi, XSS și path traversal sunt blocate de WAF.
demo-5-mysql-hardening.sh	MySQL refuză accesul nepotrivit și demonstrează TDE/audit.

6.2 Gestionarea parolelor și controlul accesului

Gestionarea parolelor este și o problemă de cultură organizațională. Oamenii aleg soluții informale atunci când procesele sigure sunt incomode. Dacă accesul la parole este clar și documentat, tentația de a copia parole în locuri nepotrivite scade. Tehnologia trebuie să sprijine comportamentul bun.

Controlul accesului trebuie să fie suficient de strict pentru protecție și suficient de practic pentru operare. Dacă accesul este prea permisiv, riscul crește. Dacă este prea complicat, oamenii caută întotdeauna scurtături (care, de cele mai multe ori, duc la dezastre!). Arhitectura propusă încearcă să păstreze acest echilibru printr-un traseu administrativ controlat.

Controlul accesului trebuie explicat clientului ca formă de protecție, nu ca obstacol birocratic. Accesul direct la toate serverele ar fi comod, dar riscant. Un traseu administrativ controlat este mai ușor de monitorizat și de justificat. Această alegere poate părea restrictivă, însă reduce expunerea.

Parolele au nevoie de un tratament asemănător. Ele nu trebuie să fie în memoria unei persoane sau într-un document obișnuit. *Key Vault* și *Ansible Vault* nu sunt doar instrumente, ci mecanisme prin care proiectul reduce dependența de obiceiuri informale.

Parolele sunt tratate drept resurse sensibile, nu ca valori de configurare obișnuite. Parolele, cheile și token-urile nu trebuie să apară în documente finale sau în fișiere versionate. Azure Key Vault și identitățile gestionate permit separarea secretelor de cod, iar Ansible Vault permite folosirea lor în configurare fără expunere directă [11].

Controlul accesului urmărește principiul privilegiului minim. Într-un proiect real, acest principiu trebuie completat cu politici clare de rotație, audit și revizuire. În lucrare, accentul este pus pe demonstrarea mecanismului și pe evitarea celor mai riscante practici: acces administrativ larg, parole în clar și servicii interne expuse inutil și periculos.

6.3 Monitorizare și răspuns operațional

Observabilitatea trebuie să lege semnalul tehnic de acțiunea umană. O alertă privind spațiul pe disc trebuie să ducă la o verificare; o alertă privind autentificări eșuate trebuie să ducă la o analiză; o indisponibilitate a site-ului trebuie să ducă la o reacție rapidă. Fără acest traseu, monitorizarea rămâne pasivă.

Într-un mediu de producție, aș completa monitorizarea cu proceduri de escaladare. Cine primește alerta, în cât timp răspunde, ce verifică prima dată și cum comunică beneficiarului? Aceste întrebări sunt operaționale, dar au o componentă umană puternică. Tehnologia semnalează problema; oamenii o gestionează.

Monitorizarea este utilă doar dacă produce acțiune. O colecție de jurnale pe care nu o citește nimeni nu îmbunătățește sistemul. De aceea, am tratat alertele ca parte a procesului operațional: ele trebuie să fie clare, proporționale și legate de responsabilități.

Răspunsul la incidente ar trebui formalizat prin *runbook*-uri. Lucrarea nu dezvoltă complet aceste *runbook*-uri, dar pregătește terenul pentru ele prin criterii de verificare, jurnale și descrierea fluxurilor tehnice. Aceasta este o direcție naturală de maturizare.

Monitorizarea transformă infrastructura dintr-un obiect mut într-un sistem care poate semnala probleme. Azure Monitor, Log Analytics (Jurnale analitice) și alertele permit observarea consumului de resurse, a stării serviciilor și a unor evenimente de securitate. Pentru un client mic, nu este necesară o platformă foarte complexă, dar este necesar un minim de vizibilitate [14].

Alertele trebuie calibrate astfel încât să fie utile, nu doar numeroase. O alertă care nu ajunge la persoana potrivită sau care se declanșează prea des devine zgomot. În proiect, monitorizarea este tratată pragmatic: suficientă pentru a demonstra maturitate operațională și pentru a susține intervenția informată.

6.4 Testare și validare

Validarea cap-coadă este importantă deoarece infrastructura poate eșua la granițele dintre etape. O imagine poate fi corectă, dar Bicep poate atașa greșit o identitate. Resursa Azure poate exista, dar Ansible poate să nu ajungă la server. Serviciul poate porni, dar monitorizarea poate să nu colecteze semnale. De aceea, testarea trebuie să traverseze straturile.

Am inclus și interpretarea problemelor întâlnite deoarece ele dau credibilitate proiectului. Un proiect fără probleme raportate pare adesea idealizat. În realitate, limitările de servicii, diferențele dintre sisteme de operare și configurările de acces apar inevitabil. Competența se vede în felul în care sunt diagnosticate și documentate.

Criteriile de maturitate atinse sunt: resurse definite prin cod, imagini standardizate, configurare idempotentă, secrete protejate, expunere controlată, monitorizare de bază și documentație de predare.

Testarea nu trebuie privită ca o etapă finală executată formal. Ea este modul prin care proiectul își verifică propriile promisiuni. Aceasta are și o funcție etică: ea protejează beneficiarul de afirmații

neverificate. Când spun că un serviciu funcționează sau că accesul este controlat, trebuie să existe o verificare. În lipsa testării, proiectul rămâne la nivel de intenție.

Am preferat teste proporționale cu scenariul. Nu ar fi realist să includ toate formele de testare *enterprise*, dar este realist să verific serviciile principale, accesul, idempotența și semnalele de securitate.

Am folosit mai multe tipuri de verificări: teste funcționale pentru servicii, teste de conectivitate, verificări de securitate, teste de idempotență și analiză a problemelor întâlnite. Problemele nu trebuie ascunse. Ele arată contactul cu realitatea tehnică. De exemplu, diferențele dintre Linux și Windows, particularitățile WinRM, durata construirii imaginilor sau limitele unor mecanisme Azure sunt aspecte care trebuie asumate și documentate.

Tip de validare	Ce verifică	Dovadă urmărită
Funcțională	Servicii web, CMS, bază de date, fișiere	Răspuns servicii și acces controlat
Securitate	Reguli NSG, acces administrativ, secrete	Porturi permise/blocate, acces prin Key Vault
Idempotență	Reluarea configurării	Rulare repetată fără schimbări nedorite
Operabilitate	Jurnale, alerte, documentație	Loguri și proceduri de predare
Reproductibilitate	Refacerea mediului	Flux Bicep/Packer/Ansible reluabil

Scriptul ``4-test-infrastructure.ps1`` verifică grupuri de resurse, VNet, subrețele, NSG-uri, Key Vault, Log Analytics, Compute Gallery, politici și VM-uri. El verifică și conectivitatea către serviciile expuse: SSH și RDP către server de acces administrativ, respectiv HTTPS către serverul web.

În planul Ansible, ``playbooks/3-verify.yml`` verifică serviciile de pe VM-uri: nginx, MySQL, SMB, xRDP și serviciile aplicațiilor. Acest tip de verificare este important deoarece o resursă Azure poate exista, dar serviciul din interiorul VM-ului poate fi oprit sau configurat incomplet.

Testele de idempotență în Bicep sunt realizate folosind `what-if` pentru a anticipa modificările înainte de o eventuală implementare și pentru a confirma că reimplementarea nu ar

produce schimbări neașteptate. Ansible contribuie prin module care urmăresc starea dorită, nu doar executarea unor comenzi [1].

6.5 Limite ale validării

Limitările sunt importante pentru că delimitează onest contribuția. Soluția nu trebuie prezentată ca produs final *enterprise*, ci ca model aplicativ matur pentru un anumit tip de client. Această precizare previne supraevaluarea rezultatului și arată unde ar trebui continuată cercetarea sau dezvoltarea. O limită importantă este lipsa exploatarei pe termen lung. Unele probleme apar abia după luni de utilizare: creșterea costurilor, acumularea de jurnale, schimbări de personal, actualizări de securitate sau cerințe noi. Lucrarea oferă o bază pentru aceste situații, nu o rezolvare completă.

Validarea prezentată nu echivalează cu un audit extern de securitate și nici cu exploatarea soluției în producție pe termen lung. Nu am măsurat comportamentul pe luni de utilizare, nu am inclus un test de penetrare complet și nu am tratat scenariile multiregionale sau cerințe de conformitate reglementată. Aceste limite sunt importante, deoarece delimitează corect contribuția lucrării.

7. Concluzii și direcții de dezvoltare

7.1 Rezultatele obținute

Dincolo de infrastructura concretă, rezultatul important este un mod de gândire. Am pornit de la întrebarea cum poate un furnizor să livreze mai responsabil pentru un client care nu are expertiză IT internă. Răspunsul nu este un singur instrument, ci combinația dintre cod, proceduri, documentație și comunicare.

Rezultatul principal este un model coerent de livrare. Am pornit de la cerințe și am ajuns la o infrastructură care poate fi descrisă, implementată și verificată. Această trecere de la nevoie la artefact tehnic este esența lucrării.

Un alt rezultat este clarificarea rolurilor instrumentelor. Bicep, Packer și Ansible nu concurează între ele, ci se completează. Înțelegerea acestei complementarități este importantă pentru proiectele reale, unde alegerea greșită a granițelor poate produce soluții greu de întreținut.

Prin această lucrare am construit și argumentat un model de infrastructură cloud automatizată pentru un client mic sau mediu. Rezultatul nu este doar o colecție de mașini virtuale, ci o metodă de livrare: cerințe analizate, arhitectură proiectată, resurse definite prin cod, imagini standardizate, servicii configurate, parole protejate, monitorizare inclusă și testare documentată.

Valoarea principală a soluției este reproductibilitatea. O infrastructură creată manual poate funcționa, dar este greu de explicat și de refăcut. O infrastructură definită prin Bicep, Packer și Ansible poate fi citită, revizuită, modificată și predată. Pentru mine, aceasta este diferența dintre o intervenție tehnică și o practică profesională.

7.2 Contribuții și valoare practică

Soluția are valoare practică mai ales pentru furnizori mici sau medii care vor să își profesionalizeze livrarea. Ea nu promite automatizare totală, ci o bază de lucru: resurse definite prin cod, imagini controlate, configurări repetabile, parole protejate și testare documentată.

Această bază poate fi adaptată. Pentru un client cu cerințe mai mari, se pot adăuga servicii gestionate, politici avansate, *backup* complet și arhitectură multiregiune. Pentru un client mai mic, anumite componente pot fi simplificate. Important este că principiile rămân aceleași.

Valoarea practică a soluției este că poate fi refolosită. Un furnizor ar putea transforma proiectul într-un șablon intern pentru clienți similari, adaptând parametrii, dimensiunile și serviciile. Această reutilizare poate reduce timpul de livrare și poate crește calitatea.

Contribuția lucrării este integrarea coerentă a tehnologiilor într-un scenariu aplicativ. Am arătat cum se combină infrastructura ca cod, imaginile standardizate, managementul configurațiilor, securitatea, monitorizarea și testarea. Am păstrat dimensiunea umană a proiectului deoarece relația client-furnizor influențează succesul livrării. O infrastructură bine proiectată reduce anxietatea beneficiarului și oferă furnizorului un mod de lucru repetabil.

Din perspectivă comercială, soluția poate deveni un șablon intern pentru furnizor. Pentru clienți similari, se pot adapta numele, dimensiunile, serviciile și secretele, păstrând logica generală. Această reutilizare nu înseamnă uniformizarea clienților, ci păstrarea unui standard de calitate.

Criterii de maturitate atinse:

Criteriu	Dovadă în proiect
Repetabilitate	Mediul poate fi recreat prin scripturi și cod versionat.
Trasabilitate	Schimbările sunt păstrate în Git și în jurnale de execuție.
Separare responsabilități	Bicep, Packer și Ansible au roluri distincte.
Securitate operațională	Secrete, NSG, întărire securitate, WAF, fail2ban și monitorizare.
Prezentabilitate	Rapoarte HTML și scripturi pentru evaluare sau client.

7.3 Limitări și dezvoltări viitoare

Dacă proiectul ar continua, prima direcție practică ar fi replicarea geografică a copiilor de siguranță pentru a asigura o redundanță a acestora în cazul unei catastrofe.

A doua direcție ar fi consolidarea lanțului de livrare. Fluxurile automatizate ar trebui protejate prin aprobări, scanări, privilegii minime și politici de ramură. Într-o lume în care infrastructura este cod, securitatea codului devine securitatea infrastructurii.

A treia direcție este rafinarea comunicării cu beneficiarul. Predarea ar putea include un document de responsabilități, un scurt ghid de solicitare a schimbărilor și un set de scenarii de incident. Aceste elemente nu sunt spectaculoase, dar reduc confuzia și întăresc încrederea.

Dezvoltările viitoare pot fi organizate pe trei direcții: securitate, operare și scalare. În zona de securitate, aș introduce scanare IaC, politici mai stricte și audit extern. În zona de operare, aș formaliza copiile de siguranță multi-zonale, *runbook*-urile și rotația frecventă a parolelor. În zona de scalare, aș evalua topologii mai complexe acolo unde situația sau cerințele ar impune-o.

O altă direcție este îmbunătățirea experienței de predare către client. Infrastructura nu este complet livrată dacă beneficiarul nu știe cum se solicită schimbări, cum se raportează incidente și ce responsabilități rămân la furnizor. Această zonă merită tratată ca parte a serviciului, nu ca anexă administrativă.

Dezvoltările viitoare ar putea include *Azure Bastion*, WinRM pe HTTPS, politici Azure mai stricte, scanare statică pentru cod IaC, teste automate de conformitate, integrare cu un sistem SIEM și

proceduri de recuperare în caz de dezastru. O direcție importantă este și formalizarea procesului de predare către client: *runbook*-uri, sesiuni de instruire, SLA-uri (Service Level Agreement) și responsabilități clar împărțite.

Lucrarea poate propune, ca direcție de maturizare, un *runbook* pentru incidentele principale. Un astfel de *runbook* nu trebuie să fie lung, ci clar: simptome, verificări, comenzi sau proceduri relevante, criterii de escaladare și pași de comunicare. În acest fel, infrastructura devine mai rezilientă nu doar prin arhitectură, ci și prin practică.

Incident posibil	Răspuns operațional
Server web indisponibil	Health check, alertă, verificare nginx, reluare playbook rol nginx dacă drift-ul este cauza.
Tentative repetate SSH	fail2ban, loguri, alertă KQL, verificare sursă și eventual blocare suplimentară.
Spațiu disc critic	Alertă Azure Monitor, analiză loguri, extindere disc sau curățare controlată.
Eșec WinRM pe Windows	Verificare runCommands/bootstrap, servicii WinRM, reguli NSG și roluri Ansible.
Certificat TLS aproape de expirare	Verificare certbot, reînnoire, test SSL și comunicare către client.
Modificare accidentală manuală	Comparare cu codul, re-rulare Bicep/Ansible, documentarea cauzei.

7.4 Concluzie finală

Privind ansamblul lucrării, consider că miza ei nu este doar tehnică, ci profesională. Am încercat să arăt că un inginer bun nu este doar cel care poate crea resurse, ci cel care poate construi un sistem inteligibil, verificabil și responsabil. Tehnologia devine mai valoroasă atunci când poate fi transmisă mai departe.

În final, infrastructura drept cod devine o formă de memorie organizațională. Ea păstrează decizii, reduce improvizația și permite altor oameni să continue munca. Aceasta este, pentru mine, legătura dintre tehnologie și dimensiunea umană a proiectului. Am înțeles, pe parcursul acestei lucrări, că tehnologia bună nu este doar cea care funcționează, ci cea care poate fi explicată, verificată, predată și îmbunătățită. În acest sens, infrastructura drept cod nu este doar o tehnică de automatizare, ci o formă de responsabilitate profesională.

Bibliografie

- [1] Microsoft. What is Bicep? Azure Resource Manager, <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview> [accesat: 16.06.2026].
- [2] S. Threxan. Azure Bicep QuickStart Pro: From JSON and ARM Templates to Advanced Deployment Techniques, CI/CD Integration, and Environment Management. GitforGits, July 2024, pp. 3-6.
- [3] J. Boero. HashiCorp Packer in Production: Efficiently Manage Sets of Images for Your Digital Transformation or Cloud Adoption Journey. Packt Publishing, June 2023, pp. 5-6.
- [4] J. Geerling. Ansible for DevOps: Server and Configuration Management for Humans. Leanpub, 2023, pp. 20-31.
- [5] A. Verdet, M. Hamdaqa, L. Da Silva, F. Khomh. Exploring Security Practices in Infrastructure as Code: An Empirical Study, arXiv:2308.03952, 2023, <https://arxiv.org/abs/2308.03952> [accesat: 16.06.2026].
- [6] A. Rahman, R. Mahdavi-Hezaveh, L. Williams. Where Are The Gaps? A Systematic Mapping Study of Infrastructure as Code Research, arXiv:1807.04872, 2018, <https://arxiv.org/abs/1807.04872> [accesat: 16.06.2026].
- [7] M. Chiari, M. De Pascalis, M. Pradella. Static Analysis of Infrastructure as Code: a Survey. IEEE 19th International Conference on Software Architecture Companion, pp. 218-225, 2022, <https://arxiv.org/abs/2206.10344> [accesat: 16.06.2026].
- [8] F. Zhao, X. Niu, S.-L. Huang, L. Zhang. Reproducing Scientific Experiment with Cloud DevOps, arXiv:1910.13397, 2019, <https://arxiv.org/abs/1910.13397> [accesat: 16.06.2026].
- [9] HashiCorp. Azure Builder, Packer integrations documentation, <https://developer.hashicorp.com/packer/integrations/hashicorp/azure/latest/components/builder/arm> [accesat: 16.06.2026].
- [10] A. Miesen. Ansible: The Practical Guide for Administrators and DevOps Teams. Rheinwerk Publishing, 2025, pp. 27, 61.
- [11] Microsoft. Azure Key Vault Overview, <https://learn.microsoft.com/en-us/azure/key-vault/general/overview> [accesat: 16.06.2026].

- [12] Microsoft. Overview of Azure Compute Gallery, <https://learn.microsoft.com/en-us/azure/virtual-machines/azure-compute-gallery> [accesat: 16.06.2026].
- [13] Ansible Community Documentation. azure.azcollection.azure_rm_inventory plugin: Azure Resource Manager inventory plugin, https://docs.ansible.com/projects/ansible/latest/collections/azure/azcollection/azure_rm_inventory.html [accesat: 16.06.2026].
- [14] Microsoft. Azure Monitor Agent Overview, <https://learn.microsoft.com/en-us/azure/azure-monitor/agents/azure-monitor-agent-overview> [accesat: 16.06.2026].

Anexa 1: Codul sursă al proiectului

Codul sursă complet al infrastructurii descrise în prezenta lucrare este disponibil public în următorul repository GitHub:

URL: <https://github.com/elvantin/disertatie>

Clonare locală:

git clone <https://github.com/elvantin/disertatie>

Structura repository-ului:

bicep/	— Definițiile infrastructurii Azure (IaC)
main.bicep, module VM/rețea/KV/monitorizare	
packer/	— Configurații golden images (Ubuntu, Windows)
ansible/	— Playbook-uri și roluri de configurare/hardening
scripts/	— Scripturi PowerShell de orchestrare (0–4)
pipelines/	— Pipeline-uri Azure DevOps CI/CD
docs/	— Documentație tehnică și diagrame arhitectură

Anexa 2. Glosar de termeni tehnici utilizați

Glosarul de mai jos reunește termenii tehnici și abrevierile folosite în lucrare, cu accent pe termenii care pot crea ambiguitate la citire. Am păstrat forma consacrată în paranteză atunci când aceasta este utilă pentru corelarea cu documentația tehnică.

Termen abreviere	Forma extinsă	Explicație în contextul lucrării
AMA	Azure Monitor Agent	Agentul Azure Monitor care colectează evenimente, metrice și jurnale de pe mașinile virtuale și le trimite către Azure Monitor/Log Analytics.
API	Application Programming Interface	Interfață prin care aplicațiile comunică între ele într-un mod controlat și documentat.
ARM	Azure Resource Manager	Platforma de management Azure prin care sunt declarate, validate și implementate resursele cloud.
Azure CLI	Azure Command-Line Interface	Interfață în linie de comandă folosită pentru administrarea și automatizarea resurselor Azure.
Azure Compute Gallery	Galerie de imagini Azure	Serviciu folosit pentru păstrarea, versionarea și reutilizarea imaginilor de mașini virtuale.
Azure DevOps	Platformă Microsoft pentru DevOps	Serviciu pentru depozite de cod, fluxuri automate, execuții controlate, artefacte și trasabilitatea schimbărilor.
Azure Key Vault	Seif de chei Azure	Serviciu Azure destinat stocării și controlului accesului la secrete, parole, chei și certificate.
Azure Monitor	Serviciu Azure de observabilitate	Serviciu pentru colectarea, interogarea și alertarea pe baza metricilor și jurnalelor operaționale.

Bicep	Limbaaj declarativ pentru Azure	Limbaaj folosit pentru descrierea infrastructurii Azure ca resurse declarative, module și parametri.
CI/CD	Continuous Integration / Continuous Delivery	Practici de integrare, validare și livrare automată a modificărilor de cod sau infrastructură.
CMS	Content Management System	Sistem de administrare a conținutului, folosit în lucrare pentru zona WordPress.
CPU	Central Processing Unit	Procesorul sistemului, monitorizat pentru consum ridicat sau comportament anormal.
DCR	Data Collection Rule	Regulă Azure Monitor care stabilește ce date colectează agentul și unde sunt trimise acestea.
DevOps	Development and Operations	Model cultural și tehnic care apropie dezvoltarea, operarea, automatizarea, validarea și feedback-ul continuu.
DNS	Domain Name System	Sistemul care asociază nume de domenii cu adrese IP.
fail2ban	Serviciu de blocare automată	Instrument care monitorizează tentative eșuate de autentificare și poate bloca temporar adrese IP.
HCL	HashiCorp Configuration Language	Limbaaj de configurare folosit în ecosistemul HashiCorp, inclusiv în șabloanele moderne Packer.
HTTPS	Hypertext Transfer Protocol Secure	Varianta securizată a HTTP, protejată prin TLS, folosită pentru accesul web public.
IaaS	Infrastructure as a Service	Model cloud în care beneficiarul folosește resurse precum mașini virtuale, rețele și stocare, păstrând responsabilități operaționale asupra sistemelor.
IaC	Infrastructure as Code	Practică prin care infrastructura este descrisă în fișiere versionabile, repetabile și verificabile.

IP	Internet Protocol	Protocol și formă de adresare folosite pentru identificarea gazdelor în rețea.
JSON	JavaScript Object Notation	Format text pentru structurarea datelor, folosit de exemplu în șabloane ARM și fișiere de configurare.
KQL	Kusto Query Language	Limbaj de interogare folosit în Log Analytics pentru analiza jurnalelor și evenimentelor.
Log Analytics	Spațiu de lucru pentru jurnale Azure	Componentă Azure Monitor unde sunt stocate și interogate jurnalele colectate.
MSI	Managed Service Identity / Managed Identity	Identitate gestionată de Azure care permite accesul controlat la resurse fără parole păstrate în cod.
nginx	Server web / reverse proxy	Server web folosit pentru publicarea și intermedierea traficului către servicii interne.
NSG	Network Security Group	Grup de securitate de rețea Azure care controlează traficul permis sau blocat către resurse/subrețele.
OCSP	Online Certificate Status Protocol	Protocol pentru verificarea stării unui certificat digital.
OS	Operating System	Sistem de operare, de exemplu Ubuntu sau Windows Server.
Packer	Instrument HashiCorp pentru imagini	Instrument folosit pentru construirea imaginilor standardizate de mașini virtuale.
PR	Public Relations	Domeniul de activitate al beneficiarului fictiv SC MEDIA SRL în scenariul lucrării.
RBAC	Role-Based Access Control	Model de control al accesului bazat pe roluri și permisiuni.
RDP	Remote Desktop Protocol	Protocol folosit pentru acces grafic la sisteme Windows sau la servicii de tip desktop la distanță.

Runbook	Procedură operațională	Set de pași documentați pentru operarea, diagnosticarea sau remedierea unor situații recurente.
SCP	Secure Copy Protocol	Protocol/comandă pentru copierea securizată a fișierelor prin SSH.
SIEM	Security Information and Event Management	Platformă pentru colectarea, corelarea și analiza evenimentelor de securitate.
SLA	Service Level Agreement	Acord privind nivelul serviciului, disponibilitatea și responsabilitățile operaționale.
SMB	Server Message Block	Protocol pentru partajarea fișierelor în rețea.
SSH	Secure Shell	Protocol securizat pentru administrarea de la distanță a sistemelor, mai ales Linux.
SSL	Secure Sockets Layer	Termen istoric pentru protecția comunicațiilor web; în practică este înlocuit de TLS.
TDE	Transparent Data Encryption	Mecanism de criptare a datelor stocate, aplicat la nivel de bază de date sau sistem compatibil.
TLS	Transport Layer Security	Protocol criptografic pentru protejarea comunicațiilor în rețea.
VM	Virtual Machine	Mașină virtuală care rulează un sistem de operare și servicii într-un mediu cloud.
VNet	Virtual Network	Rețea virtuală Azure în care sunt organizate subrețelele și comunicațiile interne.
WAF	Web Application Firewall	Firewall specializat pentru protejarea aplicațiilor web împotriva unor atacuri comune la nivel HTTP/HTTPS.
what-if	Previzualizare de implementare	Mecanism Azure/Bicep care arată schimbările probabile înainte de aplicarea unei implementări.

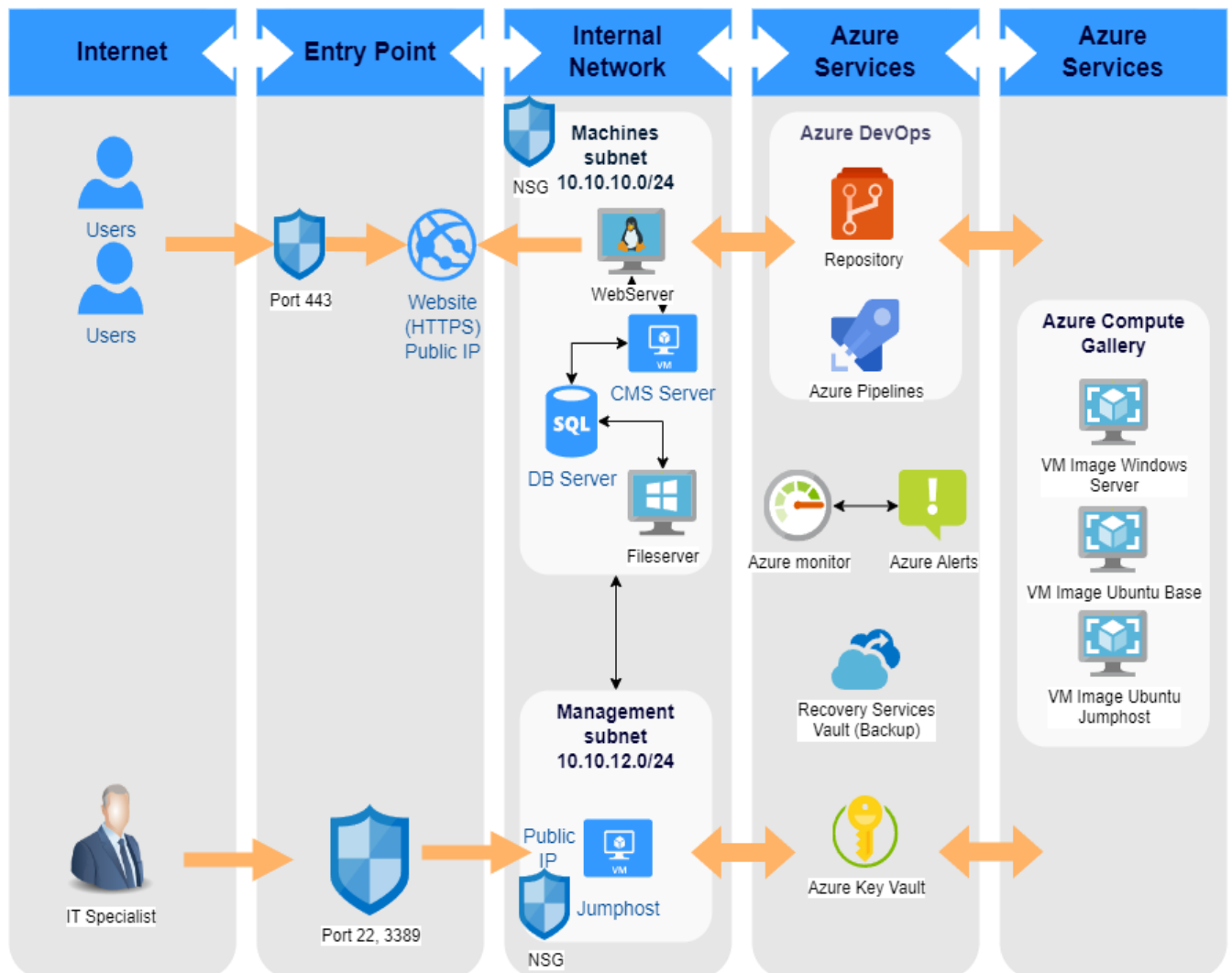
WinRM	Windows Remote Management	Protocol Microsoft pentru administrarea la distanță a sistemelor Windows, folosit de Ansible pentru Windows.
xRDP	Implementare open-source RDP	Serviciu care permite conectarea prin protocol RDP la un desktop Linux.
YAML	YAML Ain't Markup Language	Format text lizibil folosit frecvent pentru playbook-uri Ansible și fișiere de configurare.

Anexa 3. Matrice sintetică de verificare

Element verificat	Criteriu	Rezultat așteptat
Rețea	Expunere publică limitată	Doar serviciile necesare sunt accesibile public
Secrete	Fără parole în cod	Secrete gestionate prin Key Vault/Ansible Vault
Infrastructură	Definire prin cod	Resurse descrise în Bicep
Imagini	Standardizare	Imagini construite prin Packer
Configurare	Idempotență	Servicii configurate prin Ansible
Monitorizare	Jurnale și alerte	Semnale operaționale disponibile
Predare	Documentație și glosar	Clientul poate înțelege limitele și fluxul

Anexa 4: Arhitectura cloud

Arhitectura Cloud SC MEDIA SRL



Anexa 5: Firul de execuție

Fir de executie

